Evaluation Report

Group 11

1 Evaluate the Platform

For the evaluation of the platform, we simulated an incremental update of new data and measured the respective results. To simulate already existing data, we first executed an ingest pipeline to insert the old data, and we executed the update pipeline to showcase new data flowing in. All update executions showcased can be found as notebooks in the *notebooks/update_evaluation* directory.

1.1 Incremental update time

Stage	Dataset	Schema Version	Duration (Seconds)
Bronze	air_quality	2.0	19.007
Silver	air_quality	2.0	304.691
Bronze	taxi_trips	1.0	32.442
Silver	taxi_trips	1.0	255.021
Silver	weather	2.0	19.896
Bronze	weather	2.0	2.979

Table 1: Incremental Update Execution Times (Bronze to Silver)

The incremental update time (defined as the total duration to process new records from raw ingestion in the Bronze layer through validation and merging in the Silver layer) varies directly with dataset volume and transformation complexity. Based on our pipeline execution metrics, the total incremental update times were approximately 323.7 seconds for 'air_quality', 287.5 seconds for 'taxi_trips', and 22.9 seconds for the 'weather' dataset. Notice that the schema version for the taxi trips stayed the same while the others changed. This is because the new weather and air quality datasets introduced a schema change (added new columns) and thus their JSON configuration file reflected that through the updated version number and schema definition. It should be noted that the Silver processing times reflect the processing of only new data (and not any existing data), since we are doing an incremental update and only appending the new rows.

1.2 Analytical refresh time

Dataset	With recalculation (s)	Without complete recalculation (s)
air_quality_vs_demand	10.938	6.775
average_trip_distance	11.398	26.16
monthly_zone_demand	13.226	14.01
zone_weather_demand	13.716	8.415

Table 2: Recalculation overhead.

As we can see without complete rewrites and in some cases recalculation of the data products the execution time improves, except for average_trip_distance. The biggest increase can be measured on data products when we can take advantage of incremental updates.

1.3 Storage overhead introduced by the updated platform

Table 3: Storage of delta tables

File	Delta last week	Delta this week
air_quality_vs_demand	38 972	39 000
average_trip_distance_by_weather	3 673	3 770
monthly_zone_demand	11 441	11 490
zone_weather_demand_variation	7 736	7 858
integrated_taxi_trips	325 797 243	339 275 044

As we can see the sizes did not increase much. This is mostly, because we tried to avoid as much data movement as possible and did not insert records where no updates happened. This way the increase remained low across all the tables.

1.4 Validation overhead

Dataset	**Stage**	**No Validation (s)**	**With Validation (s)**	**Overhead (s)**
air_quality	Bronze	13.380	19.007	+5.627
air_quality	Silver	24.920	304.691	+279.771
taxi_trips	Bronze	24.883	32.442	+7.559
taxi_trips	Silver	25.235	255.021	+229.786
weather	Bronze	2.034	2.979	+0.945

Table 4: Validation Overhead: Execution time comparison with and without quality checks.

To accurately measure the validation overhead, we executed the incremental update pipeline twice: once bypassing the quality rules, and once fully enforcing our JSON data contracts (same table as the one in 1.1). The validation overhead (defined as the additional execution time required to check every quality rule defined in our JSON configuration) is significantly concentrated in the Silver layer (makes sense since quality rules are only checked in the Silver layer). For example, processing the ‘air_quality‘ dataset in the Silver layer took only 24.9 seconds without validation, but increased to 304.7 seconds when full quality rules were applied. This confirms that the in-memory checks introduce a substantial compute tax during the operational ingestion phase. Finally, The large validation times for the update of the air quality and taxi trips dataset can be explained by the fact that their respective sizes are really large, compared with the new weather data.

1.5 Monitoring overhead

Stage	Dataset	No Monitoring (s)	With Monitoring (s)	Overhead (s)
Bronze	air_quality	11.015	19.007	7.992
Silver	air_quality	182.334	304.691	122.357
Bronze	taxi_trips	22.795	32.442	9.647
Silver	taxi_trips	146.831	255.021	108.190
Bronze	weather	1.786	2.979	1.193
Silver	weather	10.666	19.896	9.230

Table 5: Monitoring Overhead Analysis (Bronze and Silver Stages)

The monitoring overhead represents the execution time penalty caused by actively observing the pipeline's state, such as keeping counts of processed or rejected records and reporting execution durations. Based on the measurements without monitoring enabled, the Silver layer for air_quality completed in 182.334 seconds, compared to the 304.691 seconds required when full monitoring was active. This significant addition of approximately 122 seconds of overhead occurs because calculating metrics in PySpark often requires calling action commands like .count(), which forces the engine to perform full data scans. Either way, in the Silver layer (that accounts for most monitoring costs), we notice that the overhead of monitoring is smaller than the validation overhead seen in 1.4. That is something we expected, because column and row validation is deemed a more costly operation than the monitoring of the execution.

2 Conclusions

2.1 Design Decisions that Simplified Maintenance

The design decision from Week 1 that most significantly simplified maintenance was building our ingestion and quality checks as a highly configurable, JSON-driven framework. By treating dataset definitions such as expected schemas, casting operations, and validation rules as external JSON inputs rather than hardcoded PySpark logic, we decoupled the execution engine from the dataset-specific rules. This made maintaining the platform incredibly straightforward. Adding a new data quality check or modifying an existing transformation requires zero changes to the underlying pipeline code, relying entirely on simple updates to the configuration files.

2.2 Components Requiring the Largest Modifications

Even though our configurable validation module was our most beneficial design choice, it was also the component that required the largest modifications. Initially, the framework handled bad data differently: it either logged a warning or dropped the invalid values based on JSON parameters. To achieve true production readiness, we had to heavily rewrite this module to support auditing and non-interrupted execution. We implemented logic to capture and append rejected records into Delta tables without interrupting the main pipeline, ensuring that bad data is available for inspection while the good data continues to flow into the Silver layer.

In the Silver to Gold processing, and also for the data products, we tried to avoid as much data movement and recalculation as possible. For that we had to introduce a new way of keeping track of the records and connecting it with the metadata each time data they arrived. With the introduction of the `execution_id` on record level we were able to reduce the loading times significantly (as seen by the results in this report).

2.3 Support for Future Datasets

The platform supports future datasets really well, largely due to the same metadata-driven validation framework. Whether we need to ingest a completely new dataset, process a new version of an existing source, or handle upstream schema drift (such as added or dropped columns), the core PySpark pipeline remains completely untouched. Integrating these new scenarios simply requires authoring a new JSON configuration file for the Bronze and Silver layers. The engine dynamically reads these configs to apply the correct schema constraints, isolations, and transformations, making the platform highly scalable with minimal effort.

2.4 What We Would Change in a Redesign

If we were to redesign the platform today, we would focus on automating the configuration lifecycle. Currently, introducing a new dataset requires an engineer to manually draft the JSON configuration, defining the expected schema and validation rules by hand. We would implement an automated schema-inference utility that profiles the first batch of raw data in the landing zone and autogenerates a baseline JSON contract. Additionally, rather than relying on static JSON files, We would migrate the configurations into a centralized, database-backed metadata catalog. This would allow the pipeline to dynamically fetch the latest rules via an API and provide a clear, queryable registry of all active data contracts for downstream consumers.